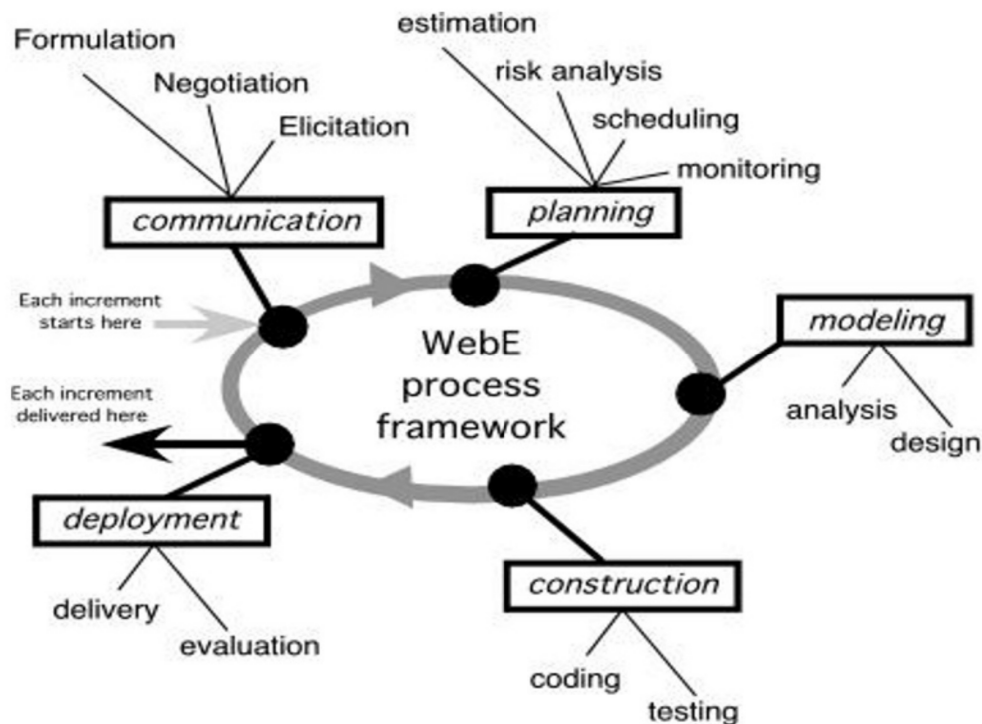


Lecture 6 Notes

Planning Cont.



Estimation (Time and Effort)

Estimation focus for most WebE projects is on macroscopic, rather than microscopic, issues. The WebE team assesses whether a planned WebApp increment can be developed with available resources according to defined schedule constraints.

■ **Two** viable (and quick) **approaches** to detailed estimation:

1. *Usage scenario-based estimation*

- Examines the usage scenarios (e.g., use cases) for the increment, compares them to previous data on average effort (E_{avg}) for previous increments. E_{avg} is the average effort (in person-days) required to deploy a usage scenario.
- Adjust estimates based on perceived complexity
- Ex: The following Figure for the fourth SafeHomeAssured.com increment, where the history for the team indicates that E_{avg} is 14 person-days. The two usage scenarios have complexity multipliers of 2.5 and 2.0, respectively. Hence, the overall effort required to implement the fourth increment is estimated to be 63 person-days. That is, if 63 person days are required to model, construct, and deploy an increment, the work could be accomplished **by one person working for**

63 days, two people working for 32 days, three people working for 21 days, and so forth. Sadly, **a linear relationship does not exist**. In reality, as more people become involved in a project, more effort is spent on communicating and coordinating (meetings, e-mail, etc.). Additional time is spent on things that have nothing to do with the project (e.g., telephone calls, coordinating the softball league), administrative issues (e.g., applying for a new health insurance plan), and other “nonproductive” work. Consequently, it may take three people 25 or more calendar days to achieve 63 person-days of effort.

Usage Scenario	E_{avg}	Complexity	Effort
Develop a layout for the space to be monitored	14	2.5	35
Get recommendations for sensor layout for my space	14	2.0	28
Totals			63

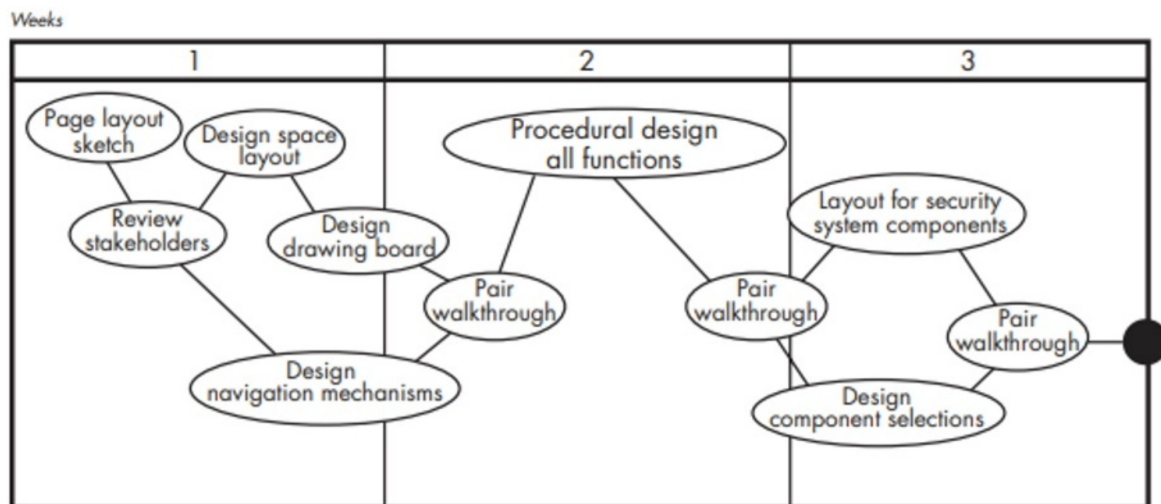
2. *Product-process table*

- First column lists, for all major WebE actions, content objects and functions for an increment
- Subsequent columns list estimates of effort (in person-days) required to perform each of the main WebE actions for each content object and function.
- Product-process table appears to be a bit more optimistic than scenario-based estimates.
- **Warning!** The relationship between effort, people and time is NOT linear.

Content and functions	Analysis	Design	Coding	Testing	Delivery	Feed-back	Total
Walls, doorways, windows	1	2	2	2	0.5	0.25	7.75
Sensors	0.5	1.5	1	1	0.25	0.25	4.5
Specify and draw walls,doorways, windows	1.25	3	3	3	1	0.25	11.5
Compute room size	0.5	1	2	1	0.5	0.25	5.25
Save/retrieve named space	0	1	1	0.5	0.5	0.25	3.25
Update/delete named space	0	1	1	0.5	0.5	0.25	3.25
Print named space	0	1	1	0.5	0.5	0.25	3.25
Recommend security hardware	0.5	3	2	2	0.5	0.25	8.25
Specify security hardware	0.5	2	2	3	0.5	0.25	8.25
Totals	4.25	15.5	15	13.5	4.75	2.25	55.25

Task Interdependencies Representation

- Individual WebE tasks have interdependencies based on their sequence.
 - Some WebE tasks or actions cannot commence until the work product produced by another is available.
 - Other tasks or actions can occur independently (can be done in parallel).
- **A task network (activity network):** is a graphic representation of the task flow for a project. The team leader should be aware of those tasks that lie on the critical path.
 - **Tasks in a critical path** that must be completed on schedule if the increment is to be completed on schedule.
 - **EX:** The following figure is for the fourth SafeHomeAssured.com increment. This increment implements a layout for the space to be monitored and recommends sensors for that space.



Monitoring (Management of quality and change)

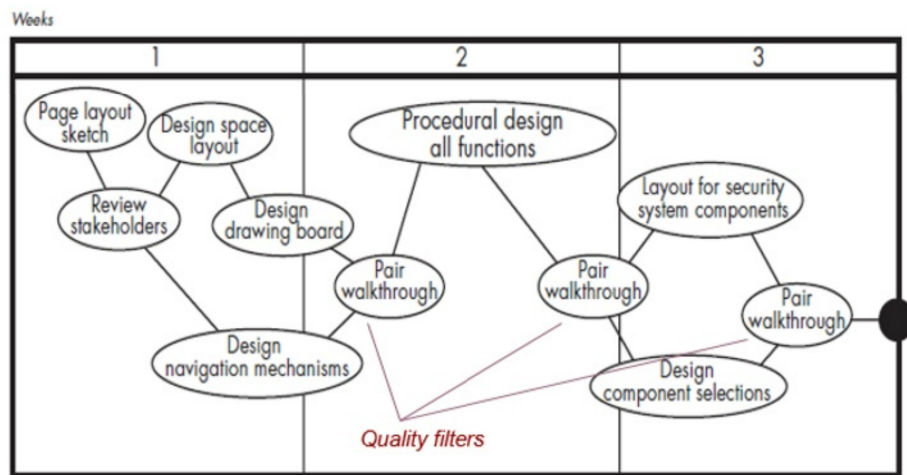
1. Managing Quality

What Quality Assurance Mechanisms Can the Team Use?

- **A thoughtful, thorough communication activity:** Communication elicits requirements from WebApp stakeholders.
- **Careful requirements gathering:** If requirements are gathered carefully, then there will be less likelihood that errors.
- **Pair walkthroughs** to assess the quality of all work products: The walkthrough participants include a Web engineer and one or more stakeholders. Two team members “pair walkthrough” review the work product as it evolves. Then, they may ask other members of the team to participate in a “team walkthrough”.

- Create a **generic checklist** that you can use to assess models
- **Use tests** to evaluate quality: tests must exercise all user requirements and at the same time examine technical aspects of the increment (e.g., architectural integrity, the correctness of a functional component or content object, and the validity of all links and navigation mechanisms).

Note That: The results of all quality assurance mechanisms assist the team when changes must be made and will also help the team to correct process weaknesses that led to a class of errors. The following figure represents **Quality Filters** (pair walkthrough) in a Task network.



What Are the Mechanics of a Pair Walkthrough?

As the walkthrough proceeds, the participants follow these guidelines:

- 1. Review the product, not the producer.** Conducted properly, the team walkthrough should leave all participants with a warm feeling of accomplishment. Errors should be pointed out gently; the intent should not be to embarrass or belittle, but rather to assist.
- 2. Set an agenda and maintain it.** A walkthrough should be kept on track and on schedule.
- 3. Limit debate and rebuttal.** When an issue is raised by a reviewer, there may not be agreement on its impact. Rather than spending time debating the question, the issue should be recorded for resolution later.
- 4. Enunciate problem areas, but don't attempt to solve every problem noted.** A walkthrough is not a problem-solving session.
- 5. Take written notes.** Notes may be entered directly into a notebook computer.
- 6. Spend enough time to uncover quality problems, but not one minute more.** In general, a team walkthrough should be completed within 60 to 90 minutes at the most.

2. Change Management

How Should Criticality and Impact of a Change Be Assessed?

1. Each change should be categorized into one of four classes:

Class 1. Content or function change that corrects a minor error or enhances local content or functionality

Class 2. Content or function change that has an impact on other content objects or functional components within the increment

Class 3. Content or function change that has a broad impact across a WebApp (e.g., major extension of functionality, significant enhancement or reduction in content, major required changes in navigation)

Class 4. Major design change (e.g., a change in interface design or navigation approach) that will be immediately noticeable to one or more categories of end users

2. Change is assessed according to the algorithm shown in the following figure:

⇒ Class 1 and 2 changes are treated informally and are handled in an agile manner.

- **For a class 1 change**, a Web engineer evaluates the impact of the change, but **no external review or documentation** is required. As the change is made, standard check-in and check-out procedures are enforced by configuration repository tools.

- **For class 2 changes**, the Web engineer reviews the impact of the change on related objects (or to ask other developers responsible for those objects to do so). If the change can be made without requiring significant changes to other objects, modification occurs without additional review or documentation.

If substantive changes are required, further evaluation and planning are necessary.

⇒ **Class 3 and 4** changes are also treated in an agile manner, but **some descriptive documentation** and more **formal review procedures** are required.

- **For a class 3 change**, A *change description* (describing the change and providing a brief assessment of the impact of the changes) is developed. The description is distributed to all members of the Web engineering team (including other stakeholders who have interest) who review it to better assess its impact.

- **For a class 4 change**, A change description is also developed, but the **review** is conducted **by all stakeholders**.

Important Notes:

- To avoid a disruption to the increment work schedule, a **change** associated with increment *n* should be **addressed only after increment *n* is initially deployed**.

■ Stakeholders must understand that **changes are not free**—they absorb **resources** and they take **time**. Therefore, the stakeholder is always left with a trade-off—work can be interrupted, and changes can be made, but delivery dates will slip, and costs will invariably escalate.

■ **Situations where the change should be made immediately:**

1. Delaying the change will result in more work.
2. The usability of the increment by end users will be severely degraded without the change.
3. Significant monetary damages will occur if the change is not made immediately.
4. Regulatory or legal requirements demand that the deployment include the change.

